

CSF2702 – Lab Class 2 –

The lab work for this session is pen and paper work, as opposed to programming. You can work in a small group for this (2 to 4 people) in order to discuss the modelling and bounce ideas around..

☐ Task 2.1

Consider a university with all the complexities of lecturing staff, courses, modules, students, enrolments, credits, etc. Attempt to model this with a UML class diagram.

You should pay particular attention to attributes, associations, and multiplicities.

Your model should end up with at least four or five classes (otherwise you are doing it wrong) and many relationships between classes.

☐ Task 2.2

Create a UML class diagram that captures the game of Snakes and Ladders.

Your model should at least capture the following:

- There are 100 squares on a board.
- Each player has a name and is represented by a piece (which has a unique colour) and is placed upon a square.
- There are snakes and ladders that connect squares (hint: consider if ladders and snakes share common behaviour).
- A dice can be rolled to generate a number between 1 and 6 (hint: model this as a class Die, with a method roll).

You should pay particular attention to attributes, associations, and multiplicities.

Your model should end up with at least four or five classes (otherwise you are doing it wrong) and many relationships between classes.

☐ Task 2.3

Choose an app that you use regularly, such as WhatsApp, Spotify, Instagram, YouTube, Deliveroo, a banking app, or a fitness tracker, etc.

Create a UML class diagram to model a small but important part of how your chosen app works. You do not need to model every feature; instead, focus on one clear area, such as messaging, playlists, posts, ordering food, or tracking workouts.

Your diagram must include:

- Between 5 and 6 classes.
- At least two or three attributes for each class.
- At least one method for each class.
- Associations between the classes.

- Appropriate multiplicities on each association, for example 1, 0..*, or 1..*.
- At least one relationship that needs careful thought, such as a many-to-many relationship or a class that connects two other classes.

For example, a music-streaming app could include the classes: **User**, **Playlist**, **Song**, **Artist**, **Album**, and **Subscription**.

Think carefully about the following questions:

- Can one user create many playlists?
- Can a song appear in many playlists?
- Can an artist create many albums?
- Does every user need a subscription?
- Which class should be responsible for actions such as `play()`, `addSong()`, or `createPlaylist()`?